

Article

SB-ObsGen: A Framework for Flyby Small Body Observation Plan Generation Using LLMs and Physics-Informed Tools

Yanzhao Li ^{1,2} , Xiaoyi Du ^{1,2}, Wenlong Niu ^{1,*} , Xiaodong Peng ¹, Yun Li ¹ and Siqi Li ¹

¹ National Space Science Center, Chinese Academy of Sciences, Beijing 100190, China; liyanzhao21@mailsucas.ac.cn (Y.L.); duxiaoyi24@mailsucas.ac.cn (X.D.); pxd@nssc.ac.cn (X.P.); liyun02@nssc.ac.cn (Y.L.); lisiqi@nssc.ac.cn (S.L.)

² University of Chinese Academy of Sciences, Beijing 101408, China

* Correspondence: niuwenlong@nssc.ac.cn

Abstract

Deep space exploration missions may encounter opportunities to visit previously unplanned small bodies, which require timely and reliable observation planning under complex engineering constraints. However, traditional manual planning is labor-intensive and difficult to adapt to multi-objective flyby scenarios, while fully autonomous systems still face reliability limitations. To address these challenges, this study proposes SB-ObsGen, a framework for flyby small-body observation plan generation that integrates large language models (LLMs) with physics-informed tools. The framework follows a Generate-and-Optimize paradigm: an Observation Plan Generation Module decomposes tasks and invokes domain-specific tools to derive feasible observation information, and a Plan Optimization Module iteratively refines the initial plan through discriminator-guided feedback. Experiments on 80 mission scenarios show that SB-ObsGen achieves strong performance across different LLM backbones, with the best configurations reaching over 85% plan consistency with reference recommendation plans. Comparative experiments against classical planning methods further show that the proposed framework is competitive on structured planning tasks while offering additional advantages in tool use, flexible constraint handling, and end-to-end planning from heterogeneous inputs. Ablation studies confirm the critical role of the Plan Optimization Module in improving final plan quality.

Keywords: deep space exploration; flyby small body exploration; observation plan generation; large language models; physics-informed tools



Academic Editors: Andrew W. H. Ip, Zhuming Bi and Kai Leung Yung

Received: 24 March 2026

Revised: 8 May 2026

Accepted: 14 May 2026

Published: 18 May 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Small bodies refer to celestial bodies that orbit the Sun but do not meet the criteria for planets or dwarf planets, including asteroids, comets, meteoroids, and other interplanetary materials [1]. They preserve abundant scientific information due to their small size, stable internal activities and low degree of evolution [2]. They are of great scientific importance for studying the origin and evolution of small bodies and the solar system, and, therefore, represent valuable exploration targets [3–5]. In recent years, the study of small bodies has become a key topic in the field of deep space exploration [6]. Small-body exploration missions typically involve long durations and large flight distances. As a result, opportunities to visit other small bodies, referred to as flybys, may arise during the mission [7]. This “opportunistic flyby” mode enables the continuous identification of potential scientific targets, thereby maximizing the overall scientific output of the mission. However, the

flight environment in deep space is extremely complex and dynamic. Unlike primary mission targets, these opportunistic targets are often not fully characterized during the initial mission design phase. Consequently, the ability to rapidly formulate and adjust observation strategies in response to emerging targets is crucial for mission success.

Generating observation plans for flyby small bodies is a sophisticated systems engineering task. It involves selecting the target small body [8], defining scientific objectives [9], and designing payload operation sequences; this paper focuses on the last of these tasks. Meanwhile, the task needs to satisfy rigorous engineering constraints—such as solar phase angles, avoidance angles for optical payloads, and power consumption limits. Traditionally, this process relies heavily on human experts who manually analyze constraints and iteratively design observation timelines [10]. While effective, this human-centric approach is time-consuming and labor-intensive, making it difficult to cope with the multi-target and variable nature of opportunity flyby scenarios. There is a pressing need for intelligent tools that can assist experts in mission analysis and accelerate the plan generation process.

With the rapid development of large language models (LLMs), representative models such as the ChatGPT series [11], LLaMA series [12], and Qwen series [13] have demonstrated strong performance in natural language processing tasks, including question answering, text summarization, and content generation [14]. They have also been increasingly applied across different domains. However, directly applying LLMs to the task of observation plan generation for flyby small bodies has the following limitations:

- (1) The LLM's grasp of domain knowledge is insufficient to meet the accuracy requirements of small body exploration missions [15]. The observation plan generation task requires diverse domain knowledge. An inaccurate result at any stage will affect the final outcome of the entire task.
- (2) The deep reasoning ability of LLMs is still limited. They may overlook key elements in long-context situations. The observation plan generation task is highly complex, involving numerous calculations, constraints, and optimizable items. It is difficult for an LLM to directly provide an optimal result. Therefore, using LLMs directly cannot meet the task requirements.

To address the LLM's insufficient domain knowledge and deep reasoning ability, this paper explores their application in deep space exploration by proposing SB-ObsGen, a framework designed to generate observation plans for flyby small bodies. The overall framework is shown in Figure 1. It divides the small-body observation planning task into two stages: Generation and Optimization. The novelty of SB-ObsGen lies not only in adopting a general generate-and-optimize paradigm, but also in adapting this paradigm to the physically constrained flyby small-body observation planning scenario. Specifically, the framework couples LLM-based flexible task interpretation with physics-informed tools and task-specific discriminators, so that the generated plans can be checked and refined under ephemeris-dependent visibility, solar-angle, payload, power, and attitude constraints. In the Generation stage, the framework analyzes user inputs to identify necessary constraints and calls domain-specific tools to calculate observable windows for each payload. Next, it uses the LLM to assign payload observation windows according to the requirements of the scientific objectives and determines the feasibility of each scientific objective to generate an initial observation plan. The subsequent Optimization stage uses a Basic Information Discriminator, a Constraint Discriminator, and an Optimizability Discriminator to detect basic errors, constraint violations, and optimization opportunities. If any issues are found, the framework automatically generates corresponding prompts for the LLM and performs iterative optimization to produce the final result. In summary, the contributions of this paper are as follows:

- (1) To address the application challenges of LLMs in deep space exploration, this paper proposes an observation plan generation framework that integrates LLMs with physical models for flyby small-body observation planning. The framework adopts a “generate-and-optimize” approach and adapts it to this task through physics-informed tool use and task-specific plan verification. It effectively resolves the conflict between the flexibility required to process scientific objectives and the precision required for engineering constraints, thus, providing a rapid and autonomous support method for complex small body observation plan generation.
- (2) To address the LLM’s insufficient deep reasoning ability, this paper designs an observation plan optimization method based on prompt engineering. It constructs a Basic Information Discriminator, a Constraint Discriminator and an Optimizability Discriminator. These discriminators identify errors, constraint violations, and optimization opportunities in the current output. This compensates for key information that the LLM may overlook during plan generation and improves the accuracy and practicality of its output.

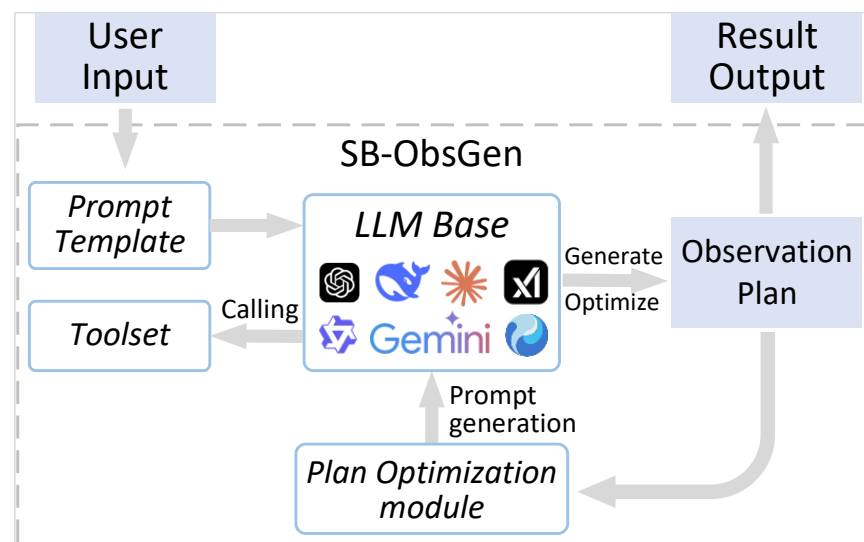


Figure 1. Overall schematic of the SB-ObsGen framework. The framework follows a generate-and-optimize workflow, in which the base LLM interprets the task input, invokes physics-informed tools to obtain key observation constraints, generates an initial plan, and iteratively refines the plan through the optimization module.

2. Related Work

To clarify the academic basis of this research, this section reviews two main research directions directly related to the study. First, observation mission planning in the aerospace field, as a classic engineering optimization problem, has seen extensive research focused on using mathematical programming and heuristic algorithms to achieve efficient scheduling of observation targets under multiple constraints. Second, the rise of Artificial Intelligence, particularly LLMs, is profoundly influencing the field of mission planning, showing strong potential for understanding natural language instructions and generating mission plans. This review aims to analyze the development directions, core methods, achievements, and limitations of these two fields, providing background and motivation for introducing LLMs into flyby small-body observation planning.

2.1. Related Work on Space Observation Mission Planning

With the rapid advancement of aerospace technology, on-orbit observation missions have become an indispensable component in fields such as deep space exploration, space

science, geographic remote sensing, and military surveillance. These missions rely on spacecraft scheduling and planning strategies to acquire observation data for specific scientific or operational objectives. Existing research mainly focuses on satellite Earth observation and multi-satellite collaborative observation, with emphasis on efficient mission scheduling and execution.

Research on satellite Earth observation mission planning began in the late 20th century. Extensive studies have investigated mission planning and re-planning for different types of ground targets, as well as optimization algorithms with improved efficiency and performance [16]. Research has examined collaborative mission planning in multi-satellite systems and explored heuristic algorithms, such as genetic algorithms and ant colony optimization, to improve scheduling performance. Early research mostly employed heuristic algorithms to solve satellite observation mission scheduling problems. For example, Wolfe et al. [17] described the satellite observation mission scheduling problem as a problem with time window constraints and introduced a novel look-ahead algorithm into the genetic algorithm, which greatly improved the performance of the algorithm. Sun et al. [18] proposed a look-ahead heuristic algorithm based on expert knowledge, which maximizes observation returns while satisfying storage and energy constraints by dynamically adjusting satellite action sequences. Similarly, Iacopino et al. [19] combined Petri-nets with hybrid ant colony algorithms to effectively handle resource conflicts in multi-satellite collaborative observations. Tangpattanakul et al. [20] addressed the mission planning problem in agile satellite Earth observation by proposing a biased random-key genetic algorithm, extending the multi-objective modeling framework based on single-objective planning methods to simultaneously optimize total mission returns and return balance.

Recent studies have further improved the efficiency of satellite observation planning in complex scenarios through various optimization algorithms. Zhou et al. [21] addressed the observation problem of multiple GEO orbit satellites by utilizing the satellites to complete observations during orbit ascent or descent, proposing a two-level optimization method to solve the observation task. Wang et al. [22] modeled constraints and used ant colony optimization for multi-satellite joint observation mission planning, verifying that it outperforms traditional single-satellite observation. Zhao et al. [23] proposed a task-clustering preprocessing method for dense target points in agile satellite mission planning, reducing solution time and improving observation efficiency. Kimoon et al. [24] used an improved Modified Dynamic Programming (MDP) method to reduce the long computation time of traditional deterministic dynamic programming in imaging mission planning for multiple agile Earth observation satellites; they established a mathematical model via Mixed Integer Linear Programming (MILP), explicitly defined the objective function and constraints, and proposed the MDP algorithm to optimize the imaging scheduling problem. Zaidi et al. [25] proposed a single-agent attention actor-critic algorithm fusing a self-attention mechanism for autonomous trajectory planning of low-thrust spacecraft, providing a new path for autonomous and efficient spacecraft mission design in multi-body gravitational environments. Wang et al. [26] addressed the two-stage decision problem of task allocation and resource scheduling in multi-satellite collaborative mission planning by constructing a bi-level planning model and proposing a Nested Bi-level Improved Genetic Algorithm (NBIGA), achieving superior optimization results and higher solution efficiency. Zhu et al. [27] proposed a Target Clusters and Dual-timeline Optimization (TCDO) framework that integrates priority-based geographical clustering with temporal-spatial coordination to effectively address attitude maneuver constraints for agile satellites during high-density, multi-priority missions. Ling et al. [28] addressed the multi-target observation requirements of high-orbit remote sensing satellites by proposing a mission planning method combining differential evolution and genetic algorithms, significantly improving obser-

vation efficiency and reducing energy consumption through an innovative “evaluation matrix” and path optimization.

Existing methods mainly focus on scenarios in which spacecraft observe different regions or different targets. However, in this mission scenario, the primary planning goal is to assign observation windows to different detection payloads for the same observation target; therefore, existing methods are difficult to transfer directly. Furthermore, although traditional methods can provide relatively precise results, they struggle with rapid adjustments in the face of unstructured inputs and dynamic environments, making it difficult to meet the extremely high requirements for timeliness and flexibility under the “explore while flying” mode.

2.2. Related Work on Large Language Models in Mission Planning

In recent years, LLMs have demonstrated strong semantic understanding and reasoning capabilities, expanding their applications from natural language processing to complex mission planning and decision-making. Compared with traditional rule-based or optimization-based methods, LLMs are better suited to handling unstructured instructions, adapting to open task environments, and generating executable action sequences from natural language interaction. Current relevant research can be roughly divided into three aspects: general agent planning, tool-augmented mission planning, and scheduling for specific aerospace scenarios.

In the field of general task and embodied intelligence planning, many studies use the common sense reasoning ability of LLMs to solve complex task decomposition problems. For robot task planning, K. Rana et al. [29] proposed the SayPlan framework, which implements semantic search through the hierarchical structure of 3D scene graphs and utilizes an iterative re-planning mechanism to verify the ability of LLMs to generate executable long-range plans from natural language instructions. For unknown or highly dynamic scenarios, Zhao et al. [30] combined LLMs with Monte Carlo Tree Search (MCTS) to propose the LLM-MCTS algorithm, using LLMs to provide heuristic strategies to guide the search direction, significantly improving planning efficiency and generalization. In addition, Tu et al. [31] proposed the AutoThink framework, which further enhances the robustness of decision-making under complex logic through adaptive reasoning mode switching. Addressing multi-agent collaborative scenarios, the hierarchical framework designed by Wang et al. [32] utilized LLMs to achieve fine-grained decomposition from teaming strategies to individual actions, verifying their interpretability advantages in collaborative tasks. These works confirm the potential of LLMs to serve as a “planning brain” for processing complex temporal logic.

Despite the powerful reasoning capabilities of LLMs, they often suffer from “hallucinations” when dealing with calculations related to specific domains; therefore, introducing external tools has become key to solving this problem. Ruan et al. [33] proposed the TPTU framework, which enables the invocation of tools through two modes: one-step agent and sequential agent, to solve complex task planning problems. In more complex game confrontation environments, Qin et al. [34] constructed the AutoPlan framework, effectively solving task decomposition and execution problems in military game scenarios by collaboratively calling external tools. Similar ideas have been applied to war-gaming; Sun et al. [35] improved reinforcement learning models by combining prior knowledge, enhancing the convergence speed of decision-making. This architecture of “large model decision-making + small model calculation” provides a theoretical basis for solving engineering planning problems that require strict physical constraints.

More recent studies have further investigated how to improve the reliability of LLM-based planning through modular agents, explicit knowledge, and verification mechanisms.

For example, Parmar et al. [36] proposed the PlanGEN framework, which introduces a model-agnostic multi-agent framework consisting of constraint, verification, and selection agents, and shows that constraint-guided iterative verification can improve inference-time planning and reasoning algorithms. Similarly, Webb et al. [37] proposed the Modular Agentic Planner (MAP), which decomposes planning into specialized LLM modules, such as error monitoring, state prediction, state evaluation, task decomposition, and task coordination, thereby improving multi-step goal-directed planning. Zhu et al. [38] proposed KnowAgent, which enhances LLM-based agents by incorporating explicit action knowledge to constrain action trajectories and mitigate planning hallucinations. These studies indicate that LLM planning benefits from modular decomposition, external knowledge, and iterative verification. However, they are mainly evaluated on general planning, reasoning, or embodied-agent benchmarks, and do not explicitly address the physical consistency, orbital-geometry constraints, payload operation constraints, and closed-loop engineering verification required in flyby small-body observation planning.

In aerospace engineering, Transformer architectures and LLMs have increasingly been introduced to optimize mission scheduling. Shi et al. [39] proposed a LLM-based bi-level scheduling algorithm for online demand processing in large-scale Earth observation satellites, combining upper-level dynamic allocation with lower-level adaptive search. Zhou et al. [40] conceptualized the training flight scheduling problem as a language modeling task, leveraging the reasoning and generalization capabilities of LLMs to generate conflict-free slot allocations. Luo et al. [41] utilized the temporal modeling ability of Transformers to propose a hierarchical prediction method for quickly generating initial plans for multi-satellite emergency observations. Xu et al. [42] combined self-attention mechanisms with Proximal Policy Optimization (PPO), achieving better performance than traditional methods in scheduling of satellite edge computing resources. Additionally, Aghzal et al. [43] proposed the PPNL benchmark to evaluate the spatio-temporal reasoning ability of LLMs, showing that current models remain limited in long-term temporal reasoning and may require fine-tuning or prompt engineering.

In summary, although recent LLM-based planning studies have demonstrated promising progress in task decomposition, modular agent collaboration, external knowledge integration, tool use, and iterative verification, their direct application to flyby small-body observation planning remains limited. Existing general LLM planners mainly focus on symbolic, embodied, or benchmark-based planning tasks, where constraints are usually expressed as textual or action-level rules rather than strict physical and engineering constraints. By contrast, aerospace-oriented intelligent planning studies mainly address Earth observation satellite scheduling, resource allocation, or emergency observation tasks, whose target characteristics and operational assumptions differ substantially from opportunistic deep-space flyby missions. Therefore, there remains a gap between flexible LLM-based planning and physically reliable observation plan generation for small-body flyby missions. Specifically, this has two main aspects: first, the lack of domain-specific physical tool integration for calculating ephemeris-dependent visibility windows, solar phase angles, solar avoidance angles, and payload observable windows; second, the lack of a targeted closed-loop optimization mechanism that can identify spacecraft engineering constraint violations and transform them into actionable refinement prompts. The SB-ObsGen framework is proposed to bridge this gap by combining LLM-based flexible task interpretation with physics-informed tools and discriminator-based iterative optimization.

3. Observation Plan Generation Framework for Flyby Small Body

3.1. Overall Framework

To improve the intelligence and autonomy of flyby small-body observation plan generation, this paper constructs the SB-ObsGen framework based on LangChain. As shown in Figure 2, SB-ObsGen adopts a two-stage scheme that integrates LLMs with domain-specific physical models.

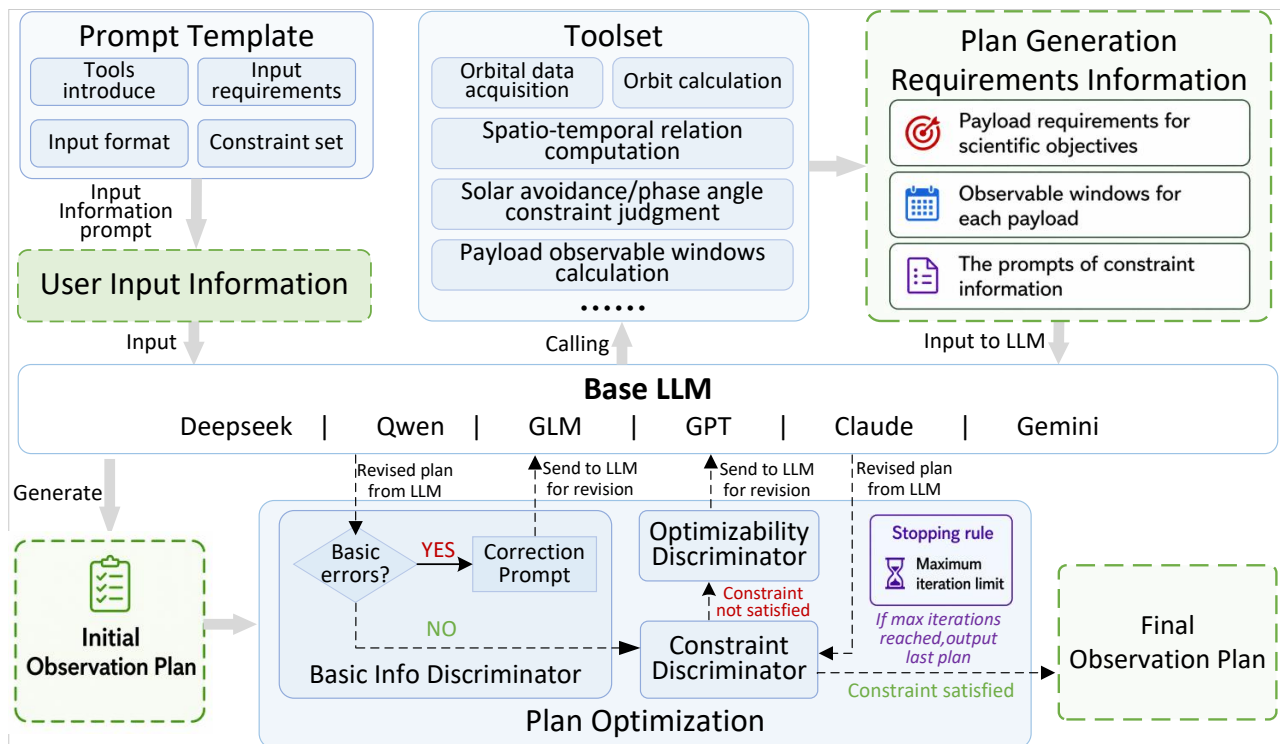


Figure 2. Detailed workflow of SB-ObsGen. The Observation Plan Generation Module constructs the initial observation plan based on structured inputs, prompt templates, and tool outputs, while the Plan Optimization Module uses basic-information, constraint, and optimizability discriminators to identify errors and generate refinement prompts.

First, the Observation Plan Generation Module uses a base LLM to analyze the user's input. This confirms key information such as task input data and constraints. It then calls relevant tools in the toolset based on the input content to perform task decomposition, and then the LLM generates an initial observation plan. To improve the rationality and feasibility of the initial observation plan, the Plan Optimization Module is used to optimize it. In the plan optimization stage, the initial plan is refined through a hierarchical discriminator-based process. The Basic Information Discriminator first acts as a gatekeeper to check whether the current plan contains basic errors, such as invalid temporal information, physical inconsistency, or incomplete satisfaction of scientific objectives. If such errors exist, the framework directly generates correction prompts and feeds them back to the LLM for revision, without entering the subsequent constraint-level optimization stage. After the plan passes the Basic Information Discriminator, the Constraint Discriminator and the Optimizability Discriminator are used to identify engineering constraint violations and potential optimization opportunities. Their outputs are converted into optimization prompts to guide the LLM in iterative refinement. The optimization process continues until the plan satisfies the required checks or the predefined maximum number of optimization iterations is reached. The latest generated plan is then output as the final observation plan. The framework consists of the following four main parts:

- (1) **Prompt Templates**
Prompt templates provide task instructions, method descriptions, tool introductions, and parameter interface specifications, ensuring that the framework receives clear and specific user input.
- (2) **Base LLM**
The framework uses mainstream LLMs (e.g., GPT-5.4, Qwen3.6-flash) as its base LLM. It calls the LLM API to implement complex tasks by using the LLM's basic capabilities.
- (3) **Toolset**
The toolset extends the capabilities of the LLM. It allows the LLM to access and process information beyond its internal knowledge, interact with other systems, or perform specialized tasks it cannot handle on its own.
- (4) **Plan Optimization Module**
The Plan Optimization Module consists of multiple discriminators, including the Basic Information Discriminator, Constraint Discriminator, and Optimizability Discriminator. In the optimization workflow, the Basic Information Discriminator is executed first to ensure the basic correctness of the current plan before constraint-level checking and soft optimization are performed. The module can identify basic information errors, constraint violations, and optimization opportunities, and constructs refinement prompts to guide the LLM.

3.2. Task Description

Before generating a flyby small-body observation plan, the spacecraft and payload parameters, target small body, scientific objectives, and spacecraft orbit are assumed to be predefined. The observation plan is, therefore, generated based on the payloads required by the scientific objectives. It combines the spatio-temporal relationship between the spacecraft and the small body, payload detection types and capabilities, and spacecraft structure to assess objective feasibility and determine the detection window for each payload.

Since the spacecraft structure is fixed, payloads may be distributed on different surfaces of the spacecraft, leading to different detection pointing directions for different payloads. Therefore, when switching between two payload observations with different pointing directions, the spacecraft must perform an attitude adjustment, and an attitude adjustment window must be reserved in the plan.

In summary, the data necessary for formulating a flyby small-body observation plan include the observable windows of each payload for the target and the list of scientific objectives. Let $\mathcal{SO} = \{so_1, \dots, so_M\}$ denote the set of scientific objectives and $\mathcal{P} = \{p_1, \dots, p_N\}$ denote the set of payloads on the spacecraft. The input requirements for scientific objectives are modeled as a set of observation tasks, denoted as $\mathcal{R}_{sci}$. Each element in $\mathcal{R}_{sci}$ is defined as a tuple representing a specific demand:

$$\mathcal{R}_{sci} = \{\langle so_i, p_j, t_{req} \rangle \mid so_i \in \mathcal{SO}, p_j \in \mathcal{P}, t_{req} \in \mathbb{R}^+\}$$

where the tuple $\langle so_i, p_j, t_{req} \rangle$ indicates that to achieve scientific objective so_i , the payload p_j is required to perform valid detection for a total duration of at least t_{req} . This formalized input structure allows the framework to handle varying mission scales and complex objective-payload mappings in a unified manner.

Since the list of scientific objectives is a mandatory input, the framework must generate payload observable windows from user-provided information before plan generation. The input content includes: (1) The accurate name, number, or orbital information of the small body; (2) Spacecraft orbit information; (3) Payload information, mainly including parameters such as payload type, power consumption, effective detection distance, etc.

Based on these necessary inputs, a preliminary observation plan can be generated, including the detection windows of the payloads. However, relevant constraints during the observation process are not fully considered, and there is room for improvement in the rationality and applicability of the observation plan. Therefore, additional inputs can be provided to represent the constraint conditions that need to be considered during the observation plan generation process, including:

(1) Rated Power of the Spacecraft: During the observation of payloads, the number of payloads operating simultaneously is limited by the rated power of the spacecraft. At any time, the total power consumption of all active payloads must not exceed the rated power of the spacecraft to ensure safe operation. Its constraint model is:

$$\sum_{i \in \mathcal{P}} W_i \cdot x_i(t) \leq P_{\max}, \quad \forall t \in [T_{\text{mission}}^{\text{start}}, T_{\text{mission}}^{\text{end}}]$$

where $\mathcal{P}$ denotes the set of all payloads. A binary variable $x_i(t) \in \{0, 1\}$ is defined to indicate whether payload i is in a working state at time t (where 1 indicates on and 0 indicates off), and W_i represents the rated power of payload i .

(2) Payload Detection Pointing Constraint: Payloads installed on different spacecraft surfaces are mutually exclusive in the time and cannot operate simultaneously. The constraint model is:

$$SF(i) \neq SF(j) \implies x_i(t) + x_j(t) \leq 1, \quad \forall t$$

where $SF(i)$ denote the installation surface of payload i .

In addition, if the payloads corresponding to the k -th observation window and the $(k+1)$ -th observation window are located on different mounting surfaces, the start time of the $(k+1)$ -th observation t_{k+1}^{start} must be later than the end time of the k -th task t_k^{end} plus the attitude adjustment duration. The constraint model is:

$$SF(p_k) \neq SF(p_{k+1}) \implies t_{k+1}^{\text{start}} - t_k^{\text{end}} \geq \Delta t_{\text{adj}}$$

(3) Attitude Adjustment Count Constraint: To save spacecraft energy, a limit on the number of attitude adjustments should be set during the observation of small bodies to minimize or restrict attitude adjustment maneuvers and extend the working life of spacecraft. Define the observation sequence as $\mathbf{O} = \{o_1, o_2, \dots, o_N\}$, where o_k represents the k -th observation window. Define the function $SF(p)$ to return the mounting surface where the payload p is located. Introduce the indicator function $\mathbb{I}(\cdot)$, which takes 1 when the condition is true and 0 otherwise. The maximum allowable number of adjustments is $N_{\text{adj}}^{\max}$. The constraint model is:

$$\sum_{k=1}^{N-1} \mathbb{I}(SF(p_k) \neq SF(p_{k+1})) \leq N_{\text{adj}}^{\max}$$

(4) Solar Phase Angle & Avoidance Angle Constraints: The solar phase angle and solar avoidance angle can be expressed as the φ angle and the θ angle in Figure 3. The phase angle constraint is used to ensure that the payload can obtain clear enough target detection data [44], while the avoidance angle constraint is used to prevent sensor damage caused by direct sunlight or high-intensity stray light [45]. The constraint model is:

Phase Angle Constraint: Let $\varphi(t)$ be the solar phase angle at time t , and $\varphi_{\max}$ be the maximum allowable phase angle:

$$\varphi(t) \leq \varphi_{\max}, \quad \forall t \in [t_{\text{start}}, t_{\text{end}}]$$

Avoidance Angle Constraint: Let $\theta(t)$ be the angle between the payload boresight and the solar vector at time t , and θ_{safe} be the minimum safe angle:

$$\theta(t) \geq \theta_{safe}, \quad \forall t \in [t_{start}, t_{end}]$$

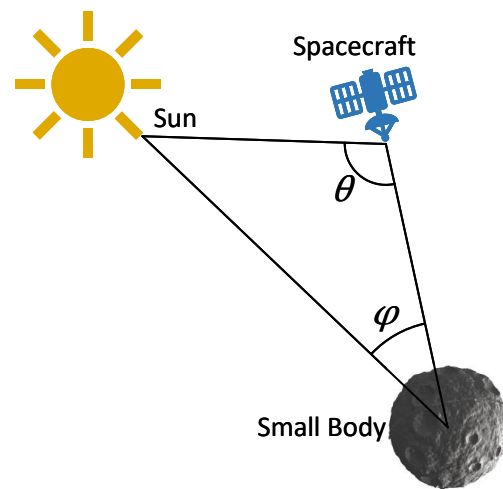


Figure 3. φ is the Sun–small body–spacecraft phase angle, θ is the angle between the payload boresight and the Sun direction, and valid observation requires $\varphi \leq \varphi_{max}$ and $\theta \geq \theta_{safe}$.

3.3. Observation Plan Generation Module

3.3.1. Prompt Design Strategy

In the observation plan generation stage, the core objective is to obtain or calculate the key information necessary for generating the observation plan, based on the initial information provided by users, and to guide the LLMs to formulate a reasonable plan. Therefore, the design of the prompt template must satisfy two requirements: information integrity and generation controllability [46]. The strategy includes the following aspects:

- (1) **Clarify the Acquisition and Calculation Methods for Key Information**
The prompt must clearly state all types of key information required to generate the observation plan. For subtasks requiring calculation or reasoning, the prompt provides calculation logic or formula guidance. If the user input is insufficient, the LLM can infer the missing basic parameters and initiate a dynamic dialogue to complete the required information.
- (2) **Instructions for Toolset Use**
To allow the LLM to effectively use external tools, the prompt must accurately declare their capability boundaries. It should provide clear instructions for each tool, including its core function, applicable scenarios, and input–output format.
- (3) **Uniform Format Constraints for Input and Output Data**
The prompt defines uniform standards for data input and output (e.g., unit systems, time formats, structured data formats). This ensures the generated content meets user expectations. It also ensures the content meets the processing requirements of downstream modules.
- (4) **Emphasize Task Constraints and Clarify Priorities**
User-provided or task-inherent constraints should be placed at the top of the prompt as high-level instructions or repeatedly emphasized. The prompt should also explain the importance of each constraint and the principles for handling conflicts, guiding the LLM to balance multiple constraints during plan generation. Differentiate between hard and soft constraints and clearly mark them in the prompt.

Through these strategies, the prompt template guides the LLM to accurately understand the task background and requirements and also minimizes information loss, format inconsistencies, and tool-calling errors. This improves the accuracy and feasibility of the generated observation plan.

3.3.2. Toolset Construction Based on Physical Models

Because the intermediate processing steps of the framework are highly specialized, conventional tools available through open-source APIs cannot fully meet the task requirements. Therefore, in addition to general-purpose tools, the toolset includes multiple domain-specific tools developed in this study. Most of these tools encapsulate various physical models from the deep space exploration domain, enabling them to calculate and process input information based on fundamental principles to obtain accurate and effective output results. The following subsections describe the functions and usage of the main tools in the toolset.

(1) Small Body Orbit Parameter Acquisition Tool

This tool is based on the authoritative database of the Minor Planet Center (MPC) under the International Astronomical Union (IAU) [47]. Using the name or number of the input small body, it automatically collects orbital data via web scraping techniques.

(2) Small Body Orbit Calculation Tool

The orbit of a small body is calculated by constructing a mechanical model and then numerically solving it [48], based on the input orbital parameter of the small body. The representation of the small body orbit is:

$$\text{Orbit}_{sb} = \{ \langle t_{orbit}^{start} + k \cdot \Delta t, x_k, y_k, z_k \rangle \mid k = 0, 1, \dots, N \}$$

where $N = \lfloor (t_{orbit}^{end} - t_{orbit}^{start}) / \Delta t \rfloor$, t_{orbit}^{start} and t_{orbit}^{end} represents the start time and end time of the orbit, and usually input by the user. Δt represents the sampling interval of the orbit.

(3) Spatio-Temporal Relationship Calculation Tool

Based on the input spacecraft orbit and small body orbit, the Spatio-Temporal Relationship Calculation Tool calculates the distance between the small body and the spacecraft at each moment to form a time-distance correspondence array, which is used to infer the detectability of each payload.

(4) Solar Phase Angle Constraint Judgment Tool

The core function of the tool is to identify the observation windows during the exploration process that satisfy this phase angle constraint ($\varphi \leq \varphi_{max}$). Firstly, calculate the solar phase angle at multiple sampling times $\varphi(t_i)$ and compare with the threshold φ_{max} . Then, using numerical interpolation, calculate the time windows where the solar phase angle meets the observation constraint.

(5) Solar Avoidance Angle Constraint Judgment Tool

The calculation method for the solar avoidance window is similar to that for the solar phase angle constraint (both involve calculating valid observation intervals), with the only difference being that the constraint is set as a minimum safe value—i.e., the angle must be greater than the set safety threshold ($\theta(t) \geq \theta_{safe}$).

(6) Payload Observable Windows Calculation Tool

This tool calculates the time windows during which each payload can perform effective detection, based on the spatio-temporal relationship between the spacecraft and the small body, as well as the payload's unobservable conditions. The final effective detection window set is :

$$W_{final} = W_{dist} \cap W_{\varphi} \cap W_{\theta}$$

where W_{dist} denotes the preliminary observable windows set calculated by the orbital spatio-temporal relationship between the spacecraft and the small body, the W_φ and W_θ denote the windows which meets the solar phase angle constraint and the solar avoidance constraint.

3.4. Plan Optimization

The Observation Plan Optimization Module adopts a hierarchical iterative refinement strategy. Its key principle is to ensure basic correctness before performing constraint-level checking and soft optimization. Therefore, the Basic Information Discriminator is first used as a gatekeeping component. If the current plan contains basic errors, the framework immediately constructs correction prompts and asks the LLM to revise the plan. The plan is evaluated by the Constraint Discriminator and Optimizability Discriminator only after it passes the basic information check. This design prevents basic factual or temporal errors from propagating into later optimization stages and improves the stability of the iterative refinement process.

3.4.1. Basic Information Discriminator

LLMs may suffer from hallucinations and factual errors in applications across various fields. In the task of designing observation plans for flyby small bodies, these issues manifest as misinterpretations of constraint and input information, leading to obvious errors in the generated observation plan. Therefore, the first step is to determine whether basic errors exist in the plan. The observation plan generated by the LLM can be defined as:

$$\mathcal{S}_{plan} = \{\langle u_n, \mathcal{T}_n \rangle\}_{n=1}^N$$

where $u_n \in \mathcal{U}$ represents the type of the n -th window, and $\mathcal{U} = \mathcal{O} \cup \mathcal{A}$ is defined as the observation windows set $\mathcal{O} = [o_1, o_2, \dots, o_q]$ and the attitude adjustment windows set $\mathcal{A} = [a_1, a_2, \dots, a_p]$ ($N = p + q$). $\mathcal{T}_n = [t_{start,n}, t_{end,n}]$ denotes the n -th task window's begin time and end time. The discriminator executes three levels of validation:

(1) Temporal Information Validity

Firstly, confirm whether the format of all times in the observation plan is correct, and then ensure that for every payload observation window, the start time precedes the end time and falls within the mission execution time $[T_{start}, T_{end}]$:

$$t_{start,n} < t_{end,n} \wedge \mathcal{T}_n \subseteq [T_{start}, T_{end}], \quad \forall n$$

(2) Physical Consistency

The planned window must be a subset of the physically observable windows calculated by the toolset. Let $\mathcal{W}_{final}(p)$ be the set of valid detection intervals for payload p , and p_n denotes the payload used in the n -th observation window. The constraint is:

$$\mathcal{T}_n \subseteq \mathcal{W}_{final}(p_n)$$

Any violation indicates that the LLM has scheduled an observation during a physically impossible period.

(3) Objective Completeness

The cumulative observation duration is verified to ensure it meets the scientific objectives requirements defined in $\mathcal{R}_{sci}$. Let $T_{total}(p)$ denote the total observation duration for payload p . Consequently, the plan is deemed complete only if:

$$T_{total}(p) \geq t_{req}, \quad \forall \langle \cdot, p, t_{req} \rangle \in \mathcal{R}_{sci}$$

If any of these conditions are not met, the Basic Information Discriminator constructs a structured prompt detailing the specific error (e.g., “Payload A lacks 20 min of observation”), which triggers the LLM to regenerate the plan.

3.4.2. Constraint Discriminator

The Constraint Discriminator mainly evaluates constraint satisfaction regarding spacecraft attitude, payload power consumption, and payload detection capability during the small body exploration process. The details are as follows:

(1) Establish Temporal and State Mappings

To strictly evaluate the system-level constraints, the observation plan $\mathcal{S}_{plan} = \{\omega_n\}_{n=1}^N$ is organized into a chronologically sorted sequence $\vec{\mathcal{S}} = (\omega_{(1)}, \omega_{(2)}, \dots, \omega_{(N)})$, where $t_{start,(n)} \leq t_{start,(n+1)}$. Subsequently, these discrete windows are mapped back to continuous state variables $x_i(t)$ for each payload $p_i \in \mathcal{P}$:

$$x_i(t) = \begin{cases} 1, & \exists \omega_n \in \mathcal{S}_{plan} \text{ s.t. } p_n = p_i \wedge t \in \mathcal{T}_n \\ 0, & \text{otherwise} \end{cases}$$

This mapping bridges the gap between the LLM-generated schedule and the physical constraint models.

(2) Spacecraft Attitude Constraints Verification

The attitude constraint discriminator focuses on identification of missing attitude adjustments, redundant attitude adjustments, and payloads on different surfaces observation conflict. The identification method of missing attitude adjustments is:

$$\mathcal{E}_m = \left\{ n \in [1, N-1] \mid \begin{array}{l} u_n \in \mathcal{O} \wedge u_{n+1} \in \mathcal{O} \\ \wedge SF(u_n) \neq SF(u_{n+1}) \end{array} \right\}$$

The identification method of redundant attitude adjustments is:

$$\mathcal{E}_r = \left\{ n \in [2, N-1] \mid \begin{array}{l} u_n \in \mathcal{A} \\ \wedge u_{n-1} \in \mathcal{O} \wedge u_{n+1} \in \mathcal{O} \\ \wedge SF(u_{n-1}) = SF(u_{n+1}) \end{array} \right\}$$

The identification method of different surfaces observation conflict is:

$$\mathcal{E}_c = \left\{ (i, j) \mid \begin{array}{l} u_i \in \mathcal{O} \wedge u_j \in \mathcal{O} \wedge \mathcal{T}_i \cap \mathcal{T}_j \neq \emptyset \\ \wedge SF(u_i) \neq SF(u_j) \end{array} \right\}$$

(3) Payload Power Consumption Constraints Verification

To determine whether the total power consumption of the payloads operating simultaneously meets the constraints, utilizing the state mapping $x_i(t)$, the instantaneous system power $P_{sys}(t)$ is derived as the aggregation of all active payloads:

$$P_{sys}(t) = \sum_{p_i \in \mathcal{P}} W_i \cdot x_i(t)$$

the discriminator identifies power overload intervals T_{over} :

$$\mathcal{T}_{over} = \{t \in [T_{start}, T_{end}] \mid P_{sys}(t) > P_{max}\}$$

If $T_{over} \neq \emptyset$, the payloads contributing to the overload at these moments are extracted to form the optimization prompt.

(4) Payload Detection Capability Constraints

The payload detection capability constraint mainly focuses on evaluating constraint evaluates whether the solar phase angle, avoidance angle, and payload detection distance requirements are satisfied. For a detection window of a payload, sampling is performed in 10-min units to calculate whether the distance between the detector and the detection target at each sampling moment satisfies the payload's detection distance constraint, and whether the positional relationship among the detector, the small body, and the Sun at each moment satisfies the phase angle and avoidance angle constraints.

3.4.3. Optimizability Discriminator

Unlike the Basic Information Discriminator and Constraint Discriminator, which enforce rigid “hard constraints”, the Optimizability Discriminator identifies potential optimization opportunities in the observation plan using “soft constraints”. However, due to the complex generation process of the observation plan, a theoretically beneficial optimization may not always be feasible in practice. Therefore, the output of this discriminator is formulated not as an error correction command, but as a reference or suggestion for LLMs to attempt optimization, while also retaining the autonomy of LLMs to preserve the current plan if the suggestion conflicts with hard constraints.

The Optimizability Discriminator evaluates optimization opportunities in the following aspects:

(1) Payload Observation Continuity

If a payload has multiple observation windows, the continuity of its observation windows should be maintained as much as possible subject to constraints; specifically, a payload should ideally have only one observation window to minimize the number of on/off switching cycles while maximizing the single continuous observation duration, thereby avoiding data gaps and saving on-board resources. For each payload p in the observation plan, let $\Omega_p = \{n \mid u_n = p\}$ be the set of indices for its observation windows, $\mathcal{W}_{detect}(p)$ denote the maximum continuous physically detectable window for payload p and $D(\mathcal{T}) = t_{end} - t_{start}$. An optimization opportunity for continuity exists if a payload is scheduled multiple times but its total duration fits within its continuous detectable window:

$$\mathcal{J}_{merge} = \left\{ p \mid \begin{array}{l} |\Omega_p| > 1 \\ \wedge \sum_{n \in \Omega_p} D(\mathcal{T}_n) \leq D(\mathcal{W}_{detect}(p)) \end{array} \right\}$$

If $p \in \mathcal{J}_{merge}$, the discriminator will suggest merging the fragmented windows in Ω_p into a single continuous block to reduce switching overhead.

(2) Reducing Attitude Adjustment Counts

Since the attitude adjustment process consumes energy of spacecraft, the number of attitude adjustments in the observation plan should be minimized.

Let $\mathcal{N}_{surfaces}$ denote the number of payload installing surfaces involved in the observation plan. The theoretical minimum number of attitude adjustment required to visit all these surfaces is $\mathcal{N}_{surfaces} - 1$. Let $N_{adj} = |\mathcal{A}|$ be the actual count of attitude adjustment windows in $\mathcal{S}_{plan}$. An optimization opportunity for agility is flagged if:

$$\mathcal{N}_{adj} > \mathcal{N}_{surfaces} - 1$$

When this inequality holds, the discriminator analyzes the observation plan sequence to identify interleaved observations between different surfaces (e.g., Surface A $\rightarrow$ Sur-

face B $\rightarrow$ Surface A) and prompts the LLM to regroup same-surface tasks (e.g., Surface A $\rightarrow$ Surface A $\rightarrow$ Surface B) to eliminate redundant maneuvers.

(3) Extending Observation Time

To fully utilize on-board resources, the observation time of payloads can be extended while meeting scientific objectives to obtain more observational data and enhance mission output. For each payload p , the total scheduled observation duration is:

$$T_{plan}^{total}(p) = \sum_{n \in \Omega_p} (t_{end,n} - t_{start,n})$$

Let $\mathcal{W}_{obs}(p) = \{[t_{s,m}, t_{e,m}]\}_{m=1}^M$ denote the set of all observable windows of payload p calculated by the Payload Observable Windows Calculation Tool. The total observable duration is:

$$T_{obs}^{total}(p) = \sum_{m=1}^M (t_{e,m} - t_{s,m})$$

Define the payload utilization η_p as the ratio of planned time to available time:

$$\eta_p = \frac{T_{plan}^{total}(p)}{T_{obs}^{total}(p)}$$

An optimization opportunity for observation time is identified if the utilization falls below a threshold (e.g., 50%):

$$\mathcal{J}_{extend} = \{p \in \mathcal{P}_{active} \mid \eta_p < 0.5\}$$

For any payload $p \in \mathcal{J}_{extend}$, the discriminator generates a soft-constraint prompt to selectively increase the observation duration of certain payloads. However, because data transmission, attitude adjustment, and thermal control are also required during observation, the optimized observation duration should not exceed 80% of the total observable duration, leaving sufficient time for key spacecraft operations.

In practical implementation, the iterative optimization process is controlled by a stopping mechanism. The framework terminates the optimization loop when either of the following conditions is met: (1) the current plan passes the required discriminator checks and no further effective refinement is identified; or (2) the number of optimization iterations reaches a predefined upper bound. In the second case, the latest generated plan is directly output as the final result. This mechanism prevents excessive iterations and ensures bounded computational cost.

4. Experiments

To comprehensively evaluate the proposed framework, three groups of experiments are conducted in this study. First, comparative experiments across different base LLMs are performed to assess the framework's adaptability to different models in terms of basic plan feasibility, plan consistency, tool-use capability, efficiency, and cost. Second, comparisons with classical planning methods are carried out on structured planning tasks to evaluate the feasibility, task completion quality, and runtime performance of the proposed framework. Third, ablation studies are conducted to verify the contributions of the Plan Optimization Module and the toolset to the overall framework performance.

4.1. Experimental Data

To evaluate SB-ObsGen, an evaluation dataset containing 80 test cases was constructed. The dataset is designed around reasoning-complexity grading, aiming to simulate mission

scenarios from simple to complex and systematically examine the framework's planning capability under different levels of input completeness. By controlling the abstraction level and completeness of input information, four increasing reasoning complexity levels (Level 1 to Level 4) are defined. The more primitive and abstract the input information, the more intermediate reasoning steps the framework requires to form the final executable plan, and, thus, the higher its reasoning complexity. The correspondence between input content and reasoning complexity level is shown in the Table 1.

Table 1. The input content of different reasoning complexity levels.

Reasoning Complexity	Input Content
Level 1	Observable windows of each payload, set of scientific objectives
Level 2	Observable windows of each payload, constraint information, set of scientific objectives
Level 3	Spatio-temporal relationship data between the spacecraft and the small body, spacecraft payload configuration, relevant constraint information, set of scientific objectives
Level 4	Small body number (or the orbit data of the small body), spacecraft orbit, spacecraft payload configuration, relevant constraint information, set of scientific objectives

The test data consists of ten categories, each containing eight test data entries of different complexity levels. These categories cover different scientific objectives, detection targets, payloads, and constraints. Specifically, the payload configurations and scientific objectives were derived from the Tianwen-2 mission [49], the Hayabusa2 mission [50], and the NEAR Shoemaker mission [51], while asteroids 2016 HO3, 433, 1998 KY26, 1999 JU3, and the main-belt comet 311P were selected as detection targets. The detailed payload configurations, detection targets, scientific objective sets, and simulated detector orbits are summarized in Table 2. However, complete spacecraft trajectory data for real small-body flyby missions are difficult to obtain because they are mission-specific and often not publicly available. Therefore, in this study, physically consistent simulated spacecraft trajectories are used to construct the spatio-temporal relationship between the spacecraft and the small body. This setting preserves the realism of target selection, payload configuration, and scientific objective design, while enabling systematic and controllable evaluation across different reasoning complexity levels and constraint combinations. For each test case, constraints such as solar phase angle constraints, solar avoidance angle constraints, spacecraft configurations indicating payload mounting surfaces, and payload power limits were set according to the payload parameters. For each test case, a manually constructed reference recommendation plan was used as the evaluation reference. In this plan, the observation window assigned to each payload is longer than the corresponding required observation duration. A framework-generated observation window is considered correct and feasible if it is contained within the corresponding window of the reference recommendation plan.

4.2. Evaluation Metrics

Since this paper includes three types of experiments with different evaluation objectives, the adopted metrics are organized according to the purpose of each experiment. The comparative experiments across different base LLMs mainly evaluate the framework's model adaptability, tool-use capability, efficiency, and cost. The comparative experiments with classical planning methods focus on final plan feasibility, task completion quality, and runtime efficiency. The ablation studies use the same metrics as the model comparison experiments, except that the toolset ablation does not include the tool-selection metric.

Table 2. The payload configuration, detection targets, scientific objective sets and simulation detector orbit of test data.

No.	Exploration Mission	Payload Configuration	Detection Target	Set of Scientific Objectives	Orbit Simulation Design
1	TianWen-2	(1) Asteroid Visible and Infrared Imaging Spectrometer (2) Asteroid Thermal Emission Spectrometer (3) Multispectral Camera (4) Asteroid Medium Angle Camera	Asteroid 2023 JJ2	A total of 7 scientific objectives: Determination of small body orbital parameters; Determination of small body rotation parameters; Determination of small body shape and size parameters;	Orbit Begin time: 25 May 2026 00:00:00 Approach Phase: 9240 min Detection Phase: 24,840 min Departure Phase: 9120 min
2		(5) Asteroid Core Scan Radar (6) Asteroid Magnetometer (7) Charged and Neutral Particle Analyzers		A total of 9 scientific objectives: Study on small body surface fine morphology; Study on the long-term orbital evolution of asteroids and the influence mechanism of the Yarkovsky effect; Analysis of the asteroid YORP effect;	Orbit Begin time: 1 Aug 2026 00:00:00 Approach Phase: 3600 min Detection Phase: 15,240 min Departure Phase: 9960 min
3		(8) Asteroid Dust and Volatiles Analyzer (9) Asteroid Narrow Angle Sensor (10) Asteroid Laser Detection And Ranging	Asteroid 2016HO3	A total of 10 scientific objectives: Study on the interaction between solar wind and small body surface materials; Study on the characteristics of particles surrounding the small body; Study on the dynamic evolution process of solar wind along the flight path;	Orbit Begin time: 1 Aug 2026 00:00:00 Approach Phase: 3600 min Detection Phase: 15,240 min Departure Phase: 9960 min
4				A total of 7 scientific objectives: Determination of small body orbital parameters; Determination of small body rotation parameters; Determination of small body shape and size;	Orbit Begin time: 1 Aug 2026 00:00:00 Approach Phase: 3600 min Detection Phase: 15,240 min Departure Phase: 9960 min
5			Comet 311P	A total of 5 scientific objectives: Study on the formation mechanism of main-belt comet surface materials; Analysis of main-belt comet water characteristics and their origins; Study on main-belt comet organic characteristics and their origins;	Orbit Begin time: 1 Oct 2026 00:00:00 Approach Phase: 16,440 min Detection Phase: 54,180 min Departure Phase: 15,780 min
6				A total of 6 scientific objectives: Study on the formation mechanism of main-belt comet surface materials; Analysis of main-belt comet water characteristics and their origins; Study on main-belt comet organic characteristics and their origins;	Orbit Begin time: 1 May 2028 00:00:00 Approach Phase: 16,440 min Detection Phase: 54,180 min Departure Phase: 15,780 min
7	NEAR Shoemaker	(1) Multi-Spectral Imager (2) NEAR IR Spectrograph (3) Three-axis Fluxgate Magnetometer (4) X-ray/Gamma-Ray Spectrometer	Asteroid 433 Eros	A total of 4 scientific objectives: Investigation of Eros' physical properties; Analysis of asteroid chemical composition; Study on the asteroid surface space environment; Monitoring of asteroid activity states	Orbit Begin time: 21 May 2001 00:00:00 Approach Phase: 5460 min Detection Phase: 32,940 min Departure Phase: 4800 min
8		(5) Laser Rangefinder		The set of scientific objectives is identical to Set 7, but the observation duration requirements for each scientific objective are different.	Orbit Begin time: 1 May 2001 00:00:00 Approach Phase: 18,720 min Detection Phase: 32,640 min Departure Phase: 4800 min

Table 2. Cont.

No.	Exploration Mission	Payload Configuration	Detection Target	Set of Scientific Objectives	Orbit Simulation Design
9	Hayabusa2	(1) Optical Navigation Cameras (2) Near-Infrared Spectrometer (3) Thermal-Infrared Imager	Asteroid 1999 JU3 (Ryugu)	A total of 4 scientific objectives: Study on surface material distribution; Investigation of organic-water alteration processes; Study on the inversion of the formation process; Study on collisional history and dynamic processes.	Orbit Begin time: 1 Jun 2018 00:00:00 Approach Phase: 45,240 min Detection Phase: 38,460 min Departure Phase: 27,180 min
10		(4) Light Detection And Ranging	Asteroid 1998 KY26	The set of scientific objectives is identical to Set 7, but the observation duration requirements for each scientific objective are different.	Orbit Begin time: 1 Jul 2031 00:00:00 Approach Phase: 16,440 min Detection Phase: 17,640 min Departure Phase: 9120 min

4.2.1. Metrics for Comparison Across Different Base LLMs

To evaluate the performance of different base LLMs within the SB-ObsGen framework, the following metrics are used:

(1) Basic Feasible Plan Rate (BFPR)

BFPR measures the proportion of plans generated by the framework for task instances that satisfy the basic feasibility conditions. A plan is counted as basically feasible only if it meets basic check requirements such as temporal validity and physical consistency with observable windows. This metric reflects whether the base language model can stably generate structurally valid candidate plans under the framework.

$$\text{BFPR} = \frac{\text{Number of Plans Passing Basic Feasibility Checks}}{\text{The Number of All Task}}$$

(2) Plan Accuracy Rate (PAR)

PAR measures the consistency between the generated observation plan and the reference recommendation plan. For each test case, if the observation window generated for a payload is contained within the corresponding window in the reference recommendation plan, that result is regarded as correct and feasible. This metric reflects the consistency of the generated plan with the manually constructed recommendation plan.

$$\text{PAR} = \frac{\sum_{j=1}^n C_j}{\sum_{j=1}^n N_j}$$

where C_j denotes the number of payload windows that are consistent with the reference recommendation plan in the j -th test case, and N_j denotes the total number of payload windows in that case.

(3) Precision of tool choosing (PTC)

PTC evaluates whether the LLM can accurately select the necessary tools from the pre-defined toolset during task decomposition. It is calculated based on the Precision and

$$\text{Precision} = |P \cap G| / |P|$$

$$\text{Recall} = |P \cap G| / |G|$$

$$\text{PTC} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(4) API Call Cost (Cost)

Cost measures the economic cost of invoking the base LLM API during the execution

of one task. It is calculated by multiplying the token consumption by the model API price. In this paper, the unit of Cost is cents(¢).

$$\text{Cost} = \text{Price}_{API} \times \text{Tokens}$$

(5) Runtime-related metrics

To evaluate the efficiency of the framework, three runtime-related metrics are used. LLM Inference Time (LIT) records the cumulative time consumed by all LLM inference calls during one task. Tool Execution Time (TET) records the cumulative time consumed by all tool calls during one task. End-to-End Runtime (Runtime) measures the total wall-clock time from task input to final plan output, reflecting the overall response efficiency of the framework in practical use.

(6) Optimization Iterations (OI)

OI denotes the number of iterative optimization rounds executed by the Plan Optimization Module for a given task. This metric reflects how many rounds of correction and refinement are required before the final plan is produced. Generally, fewer optimization iterations indicate better initial plan quality and higher convergence efficiency.

4.2.2. Metrics for Comparison with Classical Planning Methods

To compare SB-ObsGen with conventional planning methods on structured planning tasks, the following metrics are used:

(1) Feasible Plan Rate (FPR)

FPR measures the proportion of task instances for which a method generates a final plan satisfying all hard constraints. It reflects the ability of a planning method to produce executable observation plans.

$$\text{FPR} = \frac{\text{Number of feasible plans}}{\text{The Number of All Task}}$$

(2) Objective Satisfaction Ratio (OSR): OSR measures the proportion of scientific observation requirements satisfied by the generated plan. It reflects the ability of the method to fulfill the observation demands derived from the scientific objectives.

$$\text{OSR} = \frac{\text{Number of satisfied Scientific Objectives}}{\text{The Total Number of Scientific Objectives Requirements}}$$

(3) Payload Requirement Completion Rate (PRCR)

PRCR measures the extent to which the observation duration requirements of the required payloads are fulfilled. For each required payload, the completion ratio is defined as the ratio between the scheduled observation duration and the required observation duration, truncated at 1. The final PRCR is the average completion ratio over all required payloads.

$$\text{PRCR} = \frac{1}{|P_{req}|} \sum_{p \in P_{req}} \min \left(1, \frac{T_p^{plan}}{T_p^{req}} \right)$$

where P_{req} is the set of payloads required by the scientific objectives, T_p^{plan} is the scheduled observation duration of payload p , and T_p^{req} is its required observation duration.

(4) Runtime

Runtime measures the total time required by each planning method to generate its final plan.

(5) Plan Accuracy Rate (PAR)

PAR is also used in this comparison to evaluate the consistency between the generated plan and the reference recommendation plan.

4.2.3. Metrics for Ablation Studies

The ablation study of the Plan Optimization Module uses the same evaluation metrics as the base-LLM comparison experiments. In the toolset ablation study, however, the tool-selection metric is omitted because it is no longer applicable once the toolset capability is removed.

4.3. Experiment of Plan Generation Based on Different LLMs

This study evaluates the effectiveness, generalization ability, and compatibility of the proposed framework with different LLMs through a series of comparative experiments centered on replacing the foundation LLM. The framework follows a modular design principle and supports “plug-and-play” replacement of the underlying LLM, which facilitates our investigation into the relationship between model capabilities and framework performance. The experiment selected a series of representative LLMs to achieve dual evaluation goals:

- 1 Quantifying the impact of Base LLM capabilities on task performance: Comparing the performance of LLMs from different vendors under an identical framework and test set allows for clearly revealing how core capabilities, such as logical reasoning, instruction following, and domain knowledge, directly affect performance in complex task reasoning. This facilitates the definition of the capability level required for an LLM to effectively serve as the “brain” of the task generation framework.
- 2 Verifying generalization ability and compatibility of the framework: The deeper objective of this experiment is to evaluate the framework’s adaptability to various LLMs. Replacing the base LLM serves to verify the broad applicability of the framework’s modular design; specifically, the investigation focuses on whether the framework can maintain the efficient operation of structured task decomposition and tool interaction without additional optimization for specific models. This comparison confirms that the performance gains derive from the rationality of the design itself, rather than depending on the preference of a specific model.

All comparative experiments will be conducted under a unified evaluation benchmark to ensure the objectivity and comparability of the evaluation results. The sampling parameters, including temperature and top-k, were kept fixed within each model configuration. Unless otherwise specified by the API, the vendor-provided default settings were used and remained unchanged during the experimental period. To ensure sufficient iterative optimization for each mission, the framework imposes no token usage limit on the LLM. Meanwhile, for a single task case, any error or failure occurring during task execution is regarded as an unsuccessful plan generation and will be recorded statistically.

The maximum number of optimization iterations is set to 4. The model selection and the results of the comparative experiments are shown in Table 3.

Table 3. Experimental results of plan generation based on different base LLMs.

LLM	BFPR (%)	PAR (%)	PTC (%)	Cost (€)	TET (s)	LIT (s)	Runtime (s)	OI
Qwen3.6-flash	85.00	76.78	90.98	6.38	16.326	235.915	253.802	2.229
Deepseek-v4-flash	96.25	83.81	92.85	17.08	12.089	198.79	216.175	2.000
GLM-5.1	96.25	64.37	92.36	8.90	16.077	209.746	232.793	2.250
GPT-5.4	95.00	83.15	92.22	8.45	12.185	31.357	49.134	2.125
Claude-sonnet-4.6	91.25	85.48	95.96	23.90	4.998	125.112	135.569	1.879
Gemini-3.1-pro-preview	86.25	75.74	91.70	42.32	7.253	246.696	259.516	1.618

Overall, Claude-sonnet-4.6 (Anthropic, San Francisco, CA, USA) and GPT-5.4 (OpenAI, San Francisco, CA, USA) show the strongest overall performance, but with different advantages. Claude-sonnet-4.6 achieves the highest Plan Accuracy Rate (PAR) (85.48%) and Precision of Tool Choosing (PTC) (95.96%), indicating the strongest consistency with the reference recommendation plans and the best tool-selection efficiency among all evaluated models. GPT-5.4, by contrast, achieves a similarly high Basic Feasible Plan Rate (BFPR) (95.00%) and PAR (83.15%), while also showing the shortest end-to-end runtime (49.134 s). This suggests that GPT-5.4 provides the most favorable balance between plan quality and execution efficiency, whereas Claude-sonnet-4.6 is more competitive when final plan quality is prioritized.

The results further show that the different metrics do not increase monotonically with each other. In particular, BFPR, PTC, PAR, and Optimization Iterations (OI) reflect different stages of the framework and should not be interpreted as equivalent. For example, Deepseek-v4-flash (DeepSeek AI, Hangzhou, China) and GLM-5.1 (Zhipu AI, Beijing, China) both achieve a BFPR of 96.25%, indicating that they can reliably generate basically feasible candidate plans that pass the Basic Information Discriminator. However, their PAR values differ substantially, especially for GLM-5.1, whose PAR remains relatively low (64.37%). This indicates that the ability to generate a basically feasible initial plan does not necessarily translate into high final plan consistency with the reference recommendation plan. The final performance still depends on whether the model can effectively incorporate tool outputs, understand optimization prompts, and iteratively refine the plan under engineering constraints. Similarly, lower OI values indicate that fewer refinement rounds are required, but this metric should be interpreted together with final plan-quality metrics, since fewer optimization rounds do not necessarily imply better final results.

The results on tool selection provide further insight into the framework's execution process. In our experiments, the recall of tool selection is consistently close to 1 for almost all models, indicating that the required tools are generally identified correctly. Therefore, the differences in PTC mainly arise from redundant tool invocations during the optimization stage, where some models repeatedly call tools that have already been used. As a result, in the current framework, PTC reflects not only correct tool selection but also the efficiency of tool use during iterative optimization. This also suggests that future improvements in tool-call memory or tool-use control could further reduce unnecessary invocations.

From the efficiency perspective, the overall runtime differences across models are dominated mainly by LLM inference time rather than tool execution time. Tool Execution Time remains within a relatively narrow range across models, whereas LLM Inference Time varies much more significantly and largely determines the end-to-end runtime. This indicates that, in the current implementation of SB-ObsGen, the major latency bottleneck lies in model-side reasoning and generation rather than in external tool invocation. Since the present experiments are based on API calls, further improvements in response speed may be possible under future local deployment with sufficient computing resources.

Considering the above results, GPT-5.4 and Claude-sonnet-4.6 are selected as the two representative backbone models for the subsequent comparison with classical planning methods. Claude-sonnet-4.6 represents the strongest quality-oriented configuration of SB-ObsGen, while GPT-5.4 represents the most balanced configuration in terms of plan quality, execution efficiency, and cost. Using these two representative models allows the subsequent comparison to cover both the highest-quality configuration and the most practical efficiency-oriented configuration of the proposed framework.

4.4. Comparison with Classical Planning Methods

To further evaluate the effectiveness of SB-ObsGen, comparative experiments with classical planning methods are conducted on structured planning instances (40 datasets in total, covering Reasoning Complexity Levels 1 and 2), where the observable windows, scientific observation requirements, and relevant engineering constraints are predefined. This setting enables a fair comparison with conventional optimization methods, which are primarily designed for structured scheduling problems rather than the full end-to-end framework with heterogeneous inputs and tool orchestration. Thus, this experiment evaluates the structured planning core of SB-ObsGen, while intentionally excluding its additional advantages in raw-input processing, tool-based information acquisition, and heterogeneous information integration. Based on the results of the base-LLM comparison experiments, GPT-5.4 and Claude-sonnet-4.6 are selected as two representative backbone models for SB-ObsGen.

For fairness, all compared methods are evaluated on the same task instances and optimized toward the same primary objective, i.e., maximizing the number of satisfied scientific observation requirements. All methods are provided with the same observable windows, payload configurations, scientific-objective requirements, and engineering constraints, and their results are assessed using the same feasibility-checking criteria and reference recommendation plans. Therefore, the comparison should be interpreted as an evaluation of structured scheduling capability under identical input conditions, rather than as a full end-to-end comparison between LLM-based planning and classical optimization methods.

Three classical baselines are adopted: Genetic Algorithm (GA), Constraint Programming (CP), and Mixed Integer Linear Programming (MILP). The GA uses a random-key-based indirect encoding strategy, where each chromosome represents scheduling preferences rather than explicit payload states, and a decoder incrementally constructs a feasible plan under the predefined constraints. The main GA parameters are population size 200, number of generations 120, crossover probability 0.85, mutation probability 0.12, elite size 4, and random seed 42. The CP formulates the problem as a combinatorial optimization problem with requirement selection and recursive interval assignment under resource and feasibility constraints. Branch-and-bound search is employed to find the optimal schedule, with a maximum of 20,000 search nodes, at most 16 candidate intervals retained at each expansion step, and at most 64 observation segments allowed for each payload. The MILP discretizes the planning horizon at a 5 min resolution and formulates the problem using binary decision variables for payload activity, spacecraft pointing state, and requirement satisfaction. The model is solved under the predefined observable-window and engineering constraints using Gurobi. In addition to maximizing the number of satisfied requirements, a penalty term on observation fragmentation is introduced to improve schedule continuity.

The comparative experimental results of the SB-ObsGen framework and classical planning methods are shown in Table 4 and Figure 4.

Table 4. Experimental results of comparison with classical planning methods.

Baseline	FPR (%)	PAR (%)	OSR (%)	PRCR (%)	Runtime (s)
SB-ObsGen+Claude-sonnet-4.6	87.50	87.05	89.25	91.50	97.939
SB-ObsGen+GPT-5.4	80.00	85.12	85.01	86.72	30.402
GA	100.00	76.65	92.98	92.85	17.724
CP	60.00	32.65	85.24	87.05	1.826
MILP	10.00	28.73	72.79	73.54	523.002

Overall, the results show that different methods exhibit different strengths, and no single method dominates all evaluation metrics. Among the classical baselines, GA achieves

the strongest overall performance, with the highest FPR (100.00%), OSR (92.98%), and PRCR (92.85%), while also maintaining a relatively low runtime (17.724 s). This indicates that GA is a highly competitive baseline for the structured version of the observation planning problem and is well suited to the combinatorial scheduling characteristics of this task.

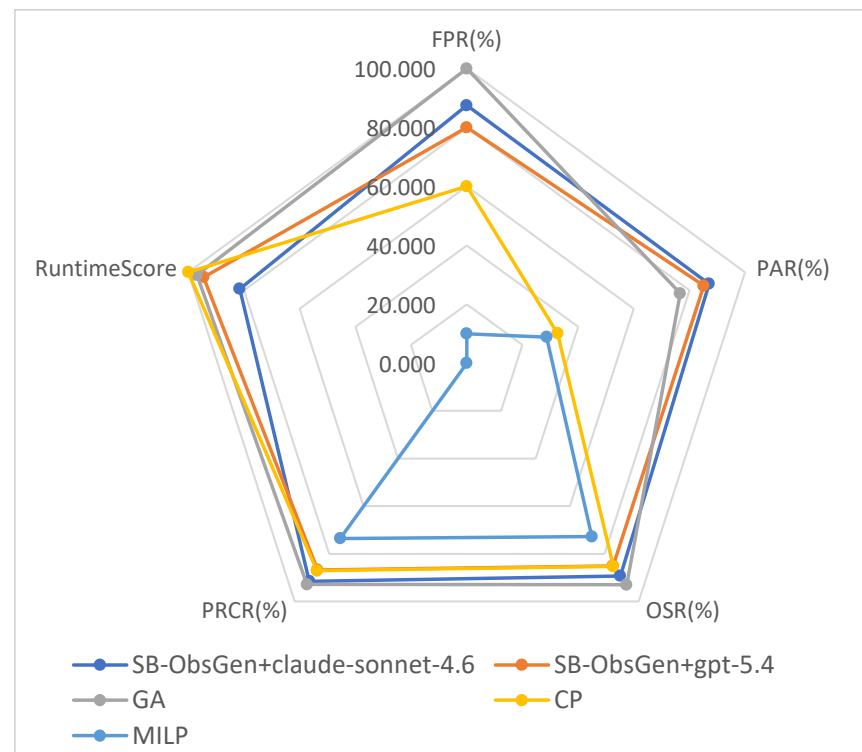


Figure 4. Radar chart of results of comparison with classical planning methods.

Among the two framework-based configurations, SB-ObsGen integrated with Claude-sonnet-4.6 achieves the highest PAR (87.05%), together with strong OSR (89.25%) and PRCR (91.50%). This suggests that the Claude-based configuration produces plans that are most consistent with the reference recommendation plans, while still maintaining strong task-completion ability. SB-ObsGen integrated with GPT-5.4 achieves slightly lower plan-quality metrics, but requires only 30.402 s, which is substantially faster than SB-ObsGen+Claude-sonnet-4.6 and much faster than MILP. Therefore, the two SB-ObsGen configurations reflect different trade-offs: Claude-sonnet-4.6 is more quality-oriented, whereas GPT-5.4 provides a more favorable balance between plan quality and runtime efficiency.

The CP and MILP baselines show weaker performance on this task. Although CP achieves the shortest runtime (1.826 s), its PAR remains low (32.65%) and its FPR is only 60.00%, indicating that fast search alone does not guarantee feasible or high-quality plans in this setting. MILP performs even less favorably, with an FPR of only 10.00%, a PAR of 28.73%, and the longest runtime (523.002 s). This suggests that the MILP formulation used in this study is less suitable for the current structured observation planning setting than GA and the proposed framework-based methods.

It should also be noted that the framework-based methods perform better in this comparison than in the previous cross-model experiments. This is because the present experiment uses structured inputs in which observable windows and relevant constraints are already available, thereby reducing the intermediate reasoning and information-completion burden on the LLM. As a result, the framework can focus more directly on plan generation and iterative optimization, leading to improved performance in both plan quality and runtime.

Although GA is highly competitive on structured planning tasks, SB-ObsGen still retains several advantages that are difficult to obtain with conventional planners alone. The proposed framework can derive planning information from relatively raw or heterogeneous inputs through tool invocation, accommodate new planning preferences or additional constraints through natural-language interaction, and tolerate minor input imperfections such as naming inconsistencies or irregular time expressions. In contrast, classical planning methods usually require pre-structured inputs and often need explicit reformulation when the problem specification changes. Therefore, SB-ObsGen should be viewed not simply as a replacement for conventional planners, but as a complementary approach that provides broader practical value in scenarios involving heterogeneous inputs, changing requirements, and interactive planning demands.

4.5. Ablation Study

To further verify the necessity of the core components within the framework and their specific contributions to overall performance, this paper also designed an Ablation Study. This part of the experiment selects the same base LLM as the comparative experiments to ensure baseline performance, and then separately “ablates” the key modules in the framework to isolate and quantify their impacts. The ablation modules include:

- (1) Ablation of the Plan Optimization Module: This experiment involves the exclusion of the Plan Optimization Module from the framework, resulting in the direct output of the initial observation plan generated by the LLM without any reflection or correction steps. Comparing the metrics before and after the removal of this module allows for a quantitative analysis of its actual utility in improving plan generation quality, correcting potential errors, and enhancing the robustness.
- (2) Ablation of the Toolset: For this experiment, the toolset is removed from the framework. By analyzing the performance gap with and without the toolset, the critical role of a rich and highly matched tool library for the framework in solving complex problems is verified, highlighting the effectiveness of the toolset constructed in this study.

Through the aforementioned ablation experiments, a deeper understanding of the collaborative mechanism among the framework’s internal components is gained, providing solid empirical support for the rationality of the framework design.

4.5.1. Plan Optimization Module Ablation Study

The results of the Plan Optimization Module ablation experiment are shown in the following Table 5.

Table 5. Experimental results of Plan Optimization Module ablation study.

LLM	BFPR (%)	PAR (%)	PTC (%)	Cost (€)	TET (s)	LIT (s)	Runtime (s)
Qwen3.6-flash	86.25	66.99	96.94	2.05	11.028	128.099	147.081
Deepseek-v4-flash	100.00	68.13	94.51	8.02	10.725	93.002	109.008
GLM-5.1	96.25	57.55	98.25	1.69	5.861	67.370	80.181
GPT-5.4	96.25	72.18	97.76	2.17	4.781	10.439	20.789
Claude-sonnet-4.6	92.50	75.50	96.24	10.64	4.785	72.956	83.191
Gemini-3.1-pro-preview	90.00	67.53	95.36	14.65	4.333	96.680	106.600

Based on the ablation results, the following observations can be made:

- (1) The Plan Optimization Module is critical for final plan quality. After this module is removed, the PAR of all tested models decreases sharply to approximately 58–76%,

- showing that the initial plans generated by the base LLMs still require substantial iterative refinement to achieve high consistency with the reference recommendation plans.
- (2) The removal of the optimization module has only limited influence on the initial-generation stage. The BFPR of most models remains relatively high, and the PTC values show no substantial degradation. This is consistent with the framework design, as these metrics mainly characterize the initial candidate-plan generation process rather than the subsequent optimization process.
 - (3) Removing the Plan Optimization Module significantly reduces the API cost and runtime-related metrics, including LLM Inference Time and End-to-End Runtime. However, these efficiency gains are achieved at the cost of a severe loss in final plan quality, which confirms that the main value of the Plan Optimization Module lies in iterative correction and refinement rather than basic plan generation.

4.5.2. Toolset Ablation Study

The results of the toolset ablation experiment are shown in the following Table 6.

Table 6. Result of toolset ablation study.

LLM	BFPR (%)	PAR (%)	LIT (s)	Runtime (s)	Cost (€)	OI
Qwen3.6-flash	41.25	74.32	300.452	305.376	8.82	2.354
Deepseek-v4-flash	47.50	84.54	263.027	269.132	27.66	2.150
GLM-5.1	48.75	66.34	240.826	248.458	20.38	2.421
GPT-5.4	47.50	85.11	50.915	59.833	19.63	2.026
Claude-sonnet-4.6	43.75	86.12	198.070	207.591	35.28	2.364
Gemini-3.1-pro-preview	45.00	75.05	290.441	301.524	62.41	2.000

The toolset is essential for maintaining the basic feasibility of generated plans. After removing the toolset, the Basic Feasible Plan Rate (BFPR) of all models drops sharply. This is mainly because about half of the test data in the dataset does not rely on tools to generate plans, so the BFPR of each model fluctuates around 50%; however, for test data requiring tools for intermediate step calculations, the framework usually fails to generate a feasible observation plan. This indicates that for professional tasks, relying solely on the built-in knowledge of LLMs is far from sufficient. Many critical steps depend on precise tools for calculation, query, or execution. Without the toolset, the model cannot form a complete, executable plan, leading to a large number of task planning failures.

Regarding the Plan Accuracy Rate (PAR), the performance of different models in the toolset ablation experiment does not show a substantial decrease compared with the full framework. This is because PAR is calculated only for successfully generated plans, and many of the remaining successful cases are those that rely mainly on the basic reasoning ability of the model rather than tool participation. Therefore, the PAR results should be interpreted together with BFPR: although the quality of some successful plans remains acceptable, the overall task success rate is greatly reduced without the toolset.

At the same time, after removing the tool calling steps, the API Call Cost, LLM Inference Time (LIT), End-to-End Runtime (Runtime) and the Optimization Iterations (OI) universally increased. The main reason is that without tool assistance, the LLMs need to perform more internal reasoning and uncertain trial-and-error attempts to compensate for the lack of external computational support, thereby consuming more computational resources.

Taken together, the two ablation experiments show that the toolset and the Plan Optimization Module play complementary roles in SB-ObsGen. The toolset provides the physical calculation and information acquisition capabilities required to handle dynamic environmental constraints, while the Plan Optimization Module improves the en-

engineering executability and reliability of generated plans through iterative correction and refinement. These results confirm that the “Tool Support + Intelligent Optimization” mechanism is essential for automated observation plan generation in flyby small-body exploration missions.

5. Discussion

The proposed SB-ObsGen framework explores the feasibility of introducing LLMs into flyby small-body observation plan generation. By combining LLM-based task understanding, physics-informed tools, and discriminator-based iterative optimization, the framework attempts to bridge flexible mission requirement interpretation and strict engineering-constraint satisfaction. The experimental results show that SB-ObsGen can process different levels of input information, invoke domain-specific tools, generate initial observation plans, and refine them according to constraint violations or optimization opportunities. This integrated workflow is particularly useful for opportunistic flyby scenarios, where mission planners may need to rapidly evaluate different scientific objectives, payload combinations, and constraint settings.

The experiments also clarify the positioning of SB-ObsGen. The comparison among different backbone LLMs shows that framework performance depends on model capabilities such as instruction following, tool use, long-context reasoning, and response efficiency. Therefore, the base LLM should be selected according to mission-analysis requirements, such as plan quality or runtime efficiency. The comparison with classical planning methods further shows that conventional planners, especially heuristic methods such as GA, remain highly competitive for the structured core of the scheduling problem. SB-ObsGen should, therefore, not be regarded as a replacement for classical planners, but as a complementary approach that provides additional advantages in natural-language interaction, heterogeneous input handling, tool-based physical calculation, and iterative refinement. Future hybrid frameworks may combine the reliability of mathematical optimization with the flexibility of LLM-based task interpretation.

Several limitations should also be acknowledged. First, the reliability of SB-ObsGen still depends partly on the capability of the base LLM. Although the physics-informed toolset and Plan Optimization Module can reduce hallucinations and identify many infeasible plan elements, they cannot completely eliminate all LLM-related errors. Human verification is, therefore, still necessary before generated plans are applied to actual mission operations.

Second, although the evaluation metrics assess feasibility, scientific-objective completion, payload requirement fulfillment, computational efficiency, and optimization iterations, they still do not constitute a complete mission-utility model. Metrics such as Objective Satisfaction Ratio and Payload Requirement Completion Rate do not yet represent weighted total science return, and the current evaluation does not fully model slew cost, momentum management, thermal effects, data volume, onboard storage, energy consumption, or robustness to trajectory and timing uncertainties. Future work should incorporate higher-fidelity spacecraft resource models, mission-specific science-return functions, uncertainty propagation, and robustness analysis.

Third, the scalability and operational deployment of the framework require further investigation. The current experiments focus mainly on single-spacecraft flyby scenarios with a limited number of payloads, scientific objectives, and constraints. Larger-scale scenarios may increase prompt length, tool calls, optimization rounds, and runtime. In addition, the current implementation is intended as a rapid mission-analysis and decision-support tool rather than a fully autonomous onboard real-time planning system, because it relies on LLM inference, external tool invocation, and iterative optimization. For stricter real-time

or onboard applications, future work should explore hierarchical planning, compact plan representations, local LLM deployment, model compression, parallel tool execution, result caching, and deterministic safety-checking mechanisms.

In addition, the experimental data are real-mission-inspired rather than fully based on actual mission trajectories. The payload configurations, target small bodies, and scientific objective settings are derived from real or representative deep-space missions, but complete spacecraft trajectory data and historical replanning records are often mission-specific and not publicly available. Therefore, physically consistent simulated spacecraft trajectories are used to construct the required spatio-temporal relationships. To improve reproducibility, the system prompt used by SB-ObsGen is provided in Appendix A, and the main experimental settings and baseline configurations are described in the experimental section. The non-sensitive experimental data, representative planning instances, and framework configurations can be made available from the corresponding author upon reasonable request. Future validation using higher-fidelity mission data, flight-like simulation environments, and expert-in-the-loop assessment will further strengthen the practical applicability of the proposed framework.

6. Conclusions

This paper proposes SB-ObsGen, an LLM-assisted framework for flyby small-body observation plan generation. The framework follows a generate-and-optimize paradigm. In the generation stage, physics-informed tools are invoked to calculate key physical and mission-related information, and the base LLM generates an initial observation plan. In the optimization stage, discriminators identify basic-information errors, constraint violations, and optimization opportunities, and then guide the LLM to iteratively refine the plan.

The experimental results demonstrate the feasibility and usefulness of the proposed framework. Comparisons across different base LLMs show that SB-ObsGen is compatible with multiple mainstream models, while comparisons with classical planning methods clarify its complementary role relative to conventional optimization and heuristic planners. The ablation studies further verify the contributions of the physics-informed toolset and the Plan Optimization Module. Overall, SB-ObsGen can provide flexible support for rapid observation-plan generation and mission analysis in opportunistic flyby small-body exploration scenarios.

Future work will focus on improving reliability, scalability, and evaluation fidelity. In particular, we will explore tighter integration with classical optimization methods, higher-fidelity spacecraft resource and uncertainty models, and broader end-to-end applications from target selection and scientific-objective recommendation to payload configuration and observation-plan generation.

Author Contributions: Conceptualization, Y.L. (Yanzhao Li) and W.N.; methodology, Y.L. (Yanzhao Li); software, Y.L. (Yanzhao Li); validation, Y.L. (Yanzhao Li); formal analysis, Y.L. (Yun Li) and X.D.; investigation, S.L. and X.D.; writing—original draft preparation, Y.L. (Yanzhao Li) and X.D.; writing—review and editing, Y.L. (Yanzhao Li) and W.N.; supervision, X.P.; funding acquisition, X.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the State Administration of Science, Technology and Industry for National Defense, PRC, under the Civil Aerospace Pre-research Project: D040103(2024).

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Appendix A. System Prompt for SB-ObsGen

You are an observation planning assistant for small-body exploration missions. Your role is to generate scientific observation plans based on payload observable windows, scientific objective data requirements, payload mounting planes, and engineering constraints.

You may call the tools provided by the system. The inputs, outputs, and functions of each tool are defined by the tool specifications. Do not fabricate tool results.

1. Task Classification

If the user only asks for an intermediate calculation, such as orbital parameters, an orbit file, a space–time relationship file, solar-constraint violation windows, payload observable windows, total window duration, or objective feasibility, complete only the requested calculation or judgment, and call the corresponding tool when needed.

If the user asks you to formulate, generate, or optimize an observation plan, follow the process below.

2. Payload Observable Window Acquisition

1. If the user has already provided payload observable windows, use them directly.
2. Otherwise, first obtain the space–time relationship between the small body and the spacecraft:
 - If a space–time relationship file is already provided, use it directly.
 - If a small-body orbit file is provided, call `space_time_cal`.
 - If small-body orbital parameters are provided, call `planet_orbit_cal`, then call `space_time_cal`.
 - If only the small-body ID is provided, call `get_orbital_parameters`, then call `planet_orbit_cal`, then call `space_time_cal`.
3. After obtaining the space–time relationship, handle solar constraints:
 - If a solar phase angle constraint is provided, call `cal_solar_phase_windows`.
 - If a solar avoidance angle constraint is provided, call `cal_solar_avoid_windows`.
 - For any solar constraint not provided, set its violation window to empty.
4. Call `cal_payload_windows` to compute the payload observable windows.

3. Observation Planning Rules

1. A scientific objective is considered achieved only when the required observation durations for all of its required payloads are satisfied.
2. Data acquired by the same payload can be shared by multiple scientific objectives, unless the user explicitly states that the data cannot be reused. Do not simply add up duration requirements from different objectives for the same payload. For example, if objective A requires 300 min of camera observation and objective B requires 500 min of camera observation, then an actual 500-min camera observation satisfies both A and B.
3. Planning priorities:
 - First, maximize the number of achieved scientific objectives.
 - If the user provides objective priorities, prioritize higher-priority objectives.
 - Then minimize the number of attitude adjustments.
 - Then keep observations by the same payload as continuous as possible.
 - If multiple plans remain, choose the one with the earliest start time.
4. Payload observations must not exceed their observable windows.

4. Attitude Adjustment Rules

1. If the user does not provide payload mounting plane information, do not plan attitude adjustments, and set `adjustment_plan` to an empty array.

2. If two consecutively observed payloads are mounted on different planes, insert an attitude adjustment window between them. If they are mounted on the same plane, no attitude adjustment is needed.
3. Use the attitude adjustment duration provided by the user. If not provided, use the default duration of 600 s.
4. During attitude adjustment, no payload may observe. Attitude adjustment windows must not overlap with observation windows.
5. If the user does not provide the initial spacecraft attitude, assume no attitude adjustment is required before the first observation segment.
5. Output Requirements

When generating a complete observation plan, output strict JSON only. Do not output a dictionary. All start_time and end_time values must use the "%Y-%m-%d %H:%M:%S" format. The duration value must be a number, with seconds as the implicit unit. Do not write units such as "seconds" or "minutes".

Output format:

```
{
  'achieved_objectives': [
    'Scientific objective 1',
    'Scientific objective 2'
  ],
  'observation_plan': [
    {
      'name': 'Payload name',
      'start_time': '2026-05-26 00:21:00',
      'end_time': '2026-05-26 01:21:00',
      'duration': 3600
    }
  ],
  'adjustment_plan': [
    {
      'name': '1st attitude adjustment',
      'start_time': '2026-05-26 01:21:00',
      'end_time': '2026-05-26 01:31:00',
      'duration': 600
    }
  ]
}
```

If no attitude adjustment is needed, output:

```
'adjustment_plan': []
```

If no scientific objective can be achieved, output:

```
{
  'achieved_objectives': [],
  'observation_plan': [],
  'adjustment_plan': [],
  'reason': 'Reason why no objective can be achieved'
}
```

6 Exception Handling

If necessary information is missing and cannot be obtained through tools, state the missing items. If non-essential information is missing, continue according to the default rules. If a tool fails, explain the reason and provide partial results based on the available information whenever possible. Do not fabricate tool outputs.

References

1. Binzel, R.P.; A'Hearn, M.; Asphaug, E.; Barucci, M.A.; Belton, M.; Benz, W.; Cellino, A.; Festou, M.C.; Fulchignoni, M.; Harris, A.W.; et al. Interiors of small bodies: Foundations and perspectives. *Planet. Space Sci.* **2003**, *51*, 443–454.
2. Nesnas, I.A.; Hockman, B.J.; Bandopadhyay, S.; Morrell, B.J.; Lubey, D.P.; Villa, J.; Bayard, D.S.; Osmundson, A.; Jarvis, B.; Bersani, M.; et al. Autonomous exploration of small bodies toward greater autonomy for deep space missions. *Front. Robot. AI* **2021**, *8*, 650885.
3. Glassmeier, K.H.; Boehnhardt, H.; Koschny, D.; Kührt, E.; Richter, I. The Rosetta mission: Flying towards the origin of the solar system. *Space Sci. Rev.* **2007**, *128*, 1–21.
4. Ramírez, N.; Sánchez-Lozano, J.M.; Peña-Asensio, E. Fuzzy Multi-Criteria Decision Framework for Asteroid Selection in Boulder Capture Missions. *Aerospace* **2025**, *12*, 800. <https://doi.org/10.3390/aerospace12090800>.
5. Yin, W.; Shu, L.; Yu, Y.; Shi, Y. Free-Vertex Tetrahedral Finite-Element Representation and Its Use for Estimating Density Distribution of Irregularly-Shaped Asteroids. *Aerospace* **2021**, *8*, 371. <https://doi.org/10.3390/aerospace8120371>.
6. Bottiglieri, C.; Piccolo, F.; Giordano, C.; Ferrari, F.; Topputo, F. Applied Trajectory Design for CubeSat Close-Proximity Operations around Asteroids: The Milani Case. *Aerospace* **2023**, *10*, 464. <https://doi.org/10.3390/aerospace10050464>.
7. Nallapu, R.T.; Thangavelautham, J. Design of spacecraft swarm flybys for planetary moon exploration. In *Proceedings of the AIAA Scitech 2020 Forum*; American Institute of Aeronautics and Astronautics: Reston, VA, USA, 2020; p. 0954.
8. Zhang, J.; Zheng, J.; Li, M. Target Pre-screening Method for Asteroid Exploration Based on Minimum Orbital Intersection Distance. *Chin. J. Space Sci.* **2022**, *42*, 973–983.
9. Li, C.; Liu, J.; Yan, W.; Feng, J.; Ren, X.; Liu, B. Overview of Scientific Objectives for Minor Planets Exploration. *J. Deep Space Explor.* **2019**, *6*, 424–436. <https://doi.org/10.15982/j.issn.2095-7777.2019.05.003>.
10. Kabir, R.H.; Peng, H.; Bai, X. Multi-step Bayesian Optimization-based intelligent task planning for an ozone observation satellite. *Acta Astronaut.* **2023**, *208*, 179–193.
11. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 27730–27744.
12. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. Llama: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971.
13. Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; Li, C.; Liu, D.; Huang, F.; Team, Q.; et al. Qwen2 technical report. *arXiv* **2024**, arXiv:2407.10671.
14. Raiaan, M.A.K.; Mukta, M.S.H.; Fatema, K.; Fahad, N.M.; Sakib, S.; Mim, M.M.J.; Ahmad, J.; Ali, M.E.; Azam, S. A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges. *IEEE Access* **2024**, *12*, 26839–26874. <https://doi.org/10.1109/ACCESS.2024.3365742>.
15. Zhou, X.; Jia, Q.; Hu, Y. Advancing General Sensor Data Synthesis by Integrating LLMs and Domain-Specific Generative Models. *IEEE Sens. Lett.* **2024**, *8*, 1–4. <https://doi.org/10.1109/LSENS.2024.3470748>.
16. Xu, M.; Hao, Y.; Bai, X.; Zhang, R.; Hu, Z.; Li, B. Research Progress on Spacecraft Earth Observation Mission Planning Technology. *J. Astronaut.* **2025**, *46*, 1501–1518. <https://doi.org/10.3873/j.issn.1000-1328.2025.08.002>.
17. Wolfe, W.J.; Sorensen, S.E. Three scheduling algorithms applied to the earth observing systems domain. *Manag. Sci.* **2000**, *46*, 148–166.
18. Sun, K.; Bai, G.; Chen, Y.; He, R.; Xing, N. Action planning for agile earth-observing satellite mission planning problem. *J. Natl. Univ. Def. Technol.* **2012**, *34*, 141.
19. Iacopino, C.; Harrison, S.; Brewer, A. Mission planning systems for commercial small-sat earth observation constellations. In *Proceedings of the 9th International Workshop on Planning and Scheduling for Space (IWPSS)*, 2015; Association for the Advancement of Artificial Intelligence: Menlo Park, CA, USA, 2015; pp. 45–52.
20. Tangpattanakul, P.; Jozefowicz, N.; Lopez, P. Multi-objective optimization for selecting and scheduling observations by agile earth observing satellites. In *Proceedings of the International Conference on Parallel Problem Solving from Nature*; Springer: Berlin/Heidelberg, Germany, 2012; pp. 112–121.
21. Zhou, Y.; Yan, Y.; Huang, X.; Yang, Y.; Zhang, H. Mission planning optimization for the visual inspection of multiple geosynchronous satellites. *Eng. Optim.* **2014**, *47*, 1543–1563. <https://doi.org/10.1080/0305215X.2014.979813>.

22. Wang, K.; Li, H. Joint observation mission planning for space exploration. In Proceedings of the 2021 2nd International Conference on Artificial Intelligence and Information Systems, ICAIIS 2021, New York, NY, USA, 28–30 May 2021. <https://doi.org/10.1145/3469213.3470342>.
23. Zhao, Y.; Du, B.; Li, S. Agile satellite mission planning via task clustering and double-layer tabu algorithm. *Comput. Model. Eng. Sci.* **2020**, *122*, 235.
24. Lee, K.; Kim, D.J.; Chung, D.W.; Lee, S. Optimal mission planning for multiple agile satellites using modified dynamic programming. *J. Aerosp. Inf. Syst.* **2024**, *21*, 279–289.
25. Zaidi, S.M.T.; Arustei, A.; Munir, A.; Dutta, A. Single-Agent Attention Actor-Critic: A Deep Reinforcement Learning-Based Solution for Low-Thrust Spacecraft Trajectory Optimization. *IEEE Trans. Aerosp. Electron. Syst.* **2025**, *61*, 11628–11643. <https://doi.org/10.1109/TAES.2025.3569615>.
26. Wang, Y.; Liu, D. A bilevel programming approach for optimizing multi-satellite collaborative mission planning. *Sensors* **2024**, *24*, 6242.
27. Zhu, G.; Zheng, Z.; Ouyang, C.; Guo, Y.; Sun, P. An Innovative Priority-Aware Mission Planning Framework for an Agile Earth Observation Satellite. *Aerospace* **2025**, *12*, 309. <https://doi.org/10.3390/aerospace12040309>.
28. Ling, L.; Zhu, Y.; Lu, Z.; Wang, J.; Wu, T.; Feng, Q. A mission planning method of high-orbit remote sensing satellites for multi-target detection. *Chin. Space Sci. Technol.* **2025**, *45*, 102.
29. Rana, K.; Haviland, J.; Garg, S.; Abou-Chakra, J.; Reid, I.; Suenderhauf, N. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv* **2023**, arXiv:2307.06135.
30. Zhao, Z.; Lee, W.S.; Hsu, D. Large language models as commonsense knowledge for large-scale task planning. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 31967–31987.
31. Tu, S.; Lin, J.; Zhang, Q.; Tian, X.; Li, L.; Lan, X.; Zhao, D. Learning When to Think: Shaping Adaptive Reasoning in R1-Style Models via Multi-Stage RL. *arXiv* **2025**, arXiv:2505.10832.
32. Wang, T.; Zhao, M.; Xu, P.; Yin, Q.; Jiao, J.; Huang, K. Decision Technology Based on Large Language Model for Wargame. *Acta Autom. Sin.* **2025**, *51*, 1205.
33. Ruan, J.; Chen, Y.; Zhang, B.; Xu, Z.; Bao, T.; Guo Qing, D.; Shiwei, S.; Mao, H.; Li, Z.; Zeng, X.; et al. Tptu: Task planning and tool usage of large language model-based ai agents. In Proceedings of the NeurIPS 2023 Foundation Models for Decision Making Workshop, Montreal, QC, Canada, 4–9 December 2023.
34. Qin, L.; Wu, W.S.; Liu, D.; Hu, Y.; Yin, Q.J.; Yang, D.S.; Wang, F.Y. Autonomous planning and processing framework for complex tasks based on large language models. *Acta Autom. Sin. A* **2024**, *50*, 862–872. <https://doi.org/10.16383/j.aas.c240088>.
35. Sun, Y.; Yuan, B.; Zhang, T.; Tang, B.; Zheng, W.; Zhou, X. Research and implementation of intelligent decision based on a priori knowledge and DQN algorithms in wargame environment. *Electronics* **2020**, *9*, 1668.
36. Parmar, M.; Liu, X.; Goyal, P.; Chen, Y.; Le, L.; Mishra, S.; Mobahi, H.; Gu, J.; Wang, Z.; Nakhosht, H.; et al. PlanGEN: A Multi-Agent Framework for Generating Planning and Reasoning Trajectories for Complex Problem Solving. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Suzhou, China, 4–9 November 2025*; Christodoulopoulos, C., Chakraborty, T., Rose, C., Peng, V., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 20640–20666. <https://doi.org/10.18653/v1/2025.emnlp-main.1042>.
37. Webb, T.; Mondal, S.S.; Momennejad, I. A brain-inspired agentic architecture to improve planning with LLMs. *Nat. Commun.* **2025**, *16*, 8633.
38. Zhu, Y.; Qiao, S.; Ou, Y.; Deng, S.; Lyu, S.; Shen, Y.; Liang, L.; Gu, J.; Chen, H.; Zhang, N. KnowAgent: Knowledge-Augmented Planning for LLM-Based Agents. In *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, NM, USA, 29 April–4 May 2025*; Chiruzzo, L., Ritter, A., Wang, L., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 3709–3732. <https://doi.org/10.18653/v1/2025.findings-naacl.205>.
39. Shi, H.; Du, Y.; Zhang, Z.; Li, L.; Wang, F. LLM Based Bi-Level Online Order Dispatching and Scheduling for Large-Scale Earth Observation Satellites. In *Proceedings of the 2025 IEEE Congress on Evolutionary Computation (CEC)*; IEEE: New York, NY, USA, 2025; pp. 1–4.
40. Zhou, W.; Wang, J.; Zhu, L.; Wang, Y.; Ji, Y. Flight Arrival Scheduling via Large Language Model. *Aerospace* **2024**, *11*, 813. <https://doi.org/10.3390/aerospace11100813>.
41. Luo, Z.; Du, C.; Chen, H.; Peng, S.; Li, J. Multi-satellite scheduling approach for emergency scenarios based on hierarchical forecasting with transformer network. *Acta Aeronaut. Astronaut. Sin.* **2021**, *42*, 464–475.
42. Xu, X.; Xia, Y.; Peng, Q.; Zhong, X.; Zhou, S.; Peng, K.; Wang, M.; Zhao, J. A Novel Self-Attention-Enhanced Multi-Neighborhood PPO Scheduling Approach for Satellite Edge Computing. In *Proceedings of the 2025 IEEE International Conference on Web Services (ICWS)*; IEEE: New York, NY, USA, 2025; pp. 1–10.
43. Aghzal, M.; Plaku, E.; Yao, Z. Can large language models be good path planners? a benchmark and investigation on spatial-temporal reasoning. *arXiv* **2023**, arXiv:2310.03249.

44. Kervin, P.W.; Hall, D.; Bolden, M.; Toth, J. Phase Angle: What is it good for. In Proceedings of the Advanced Maui Optical Space and Surveillance Technologies Conference, Maui, HI, USA, 14–17 September 2010; pp. 14–17.
45. Shi, X.; Li, X.; Wang, Y. A mission planning method for sunlight avoidance period of GEO optical remote sensing satellite. *J. Phys. Conf. Ser.* **2022**, 2364, 012056.
46. Ren, S.; Nakagawa, H.; Tsuchiya, T. Combining Prompts with Examples to Enhance LLM-Based Requirement Elicitation. In *Proceedings of the 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2024; IEEE: New York, NY, USA, 2024; pp. 1376–1381. <https://doi.org/10.1109/COMPSAC61105.2024.00181>.
47. Minor Planet Center. Available online: <https://www.minorplanetcenter.net/> (accessed on 2 December 2025)
48. Liu, L.; Ji, J. Research on the Orbital Evolution of Near-Earth Asteroids. *Acta Astron. Sin.* **2001**, 1, 75–80. <https://doi.org/10.15940/j.cnki.0001-5245.2001.01.011>.
49. Li, C.; Liu, J.; Ren, X.; Yan, W.; Zhang, Z.; Li, H.; Ouyang, Z. Scientific objectives and payloads configuration of the Tianwen-2 mission. *J. Deep Space Explor.* **2024**, 11, 304–310. <https://doi.org/10.15982/j.issn.2096-9287.2024.20230185>.
50. Tsuda, Y.; Yoshikawa, M.; Abe, M.; Minamino, H.; Nakazawa, S. System design of the Hayabusa 2—Asteroid sample return mission to 1999 JU3. *Acta Astronaut.* **2013**, 91, 356–362.
51. Prockter, L.; Murchie, S.; Cheng, A.; Krimigis, S.; Farquhar, R.; Santo, A.; Trombka, J. The NEAR shoemaker mission to asteroid 433 eros. *Acta Astronaut.* **2002**, 51, 491–500.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.